Configure software without changing source code:

Kconfig and *DeviceTree*

Copyright © 2025 Nordic Semiconductor. All rights reserved

Copyright © 2025 Nordic Semiconductor. All rights reserved

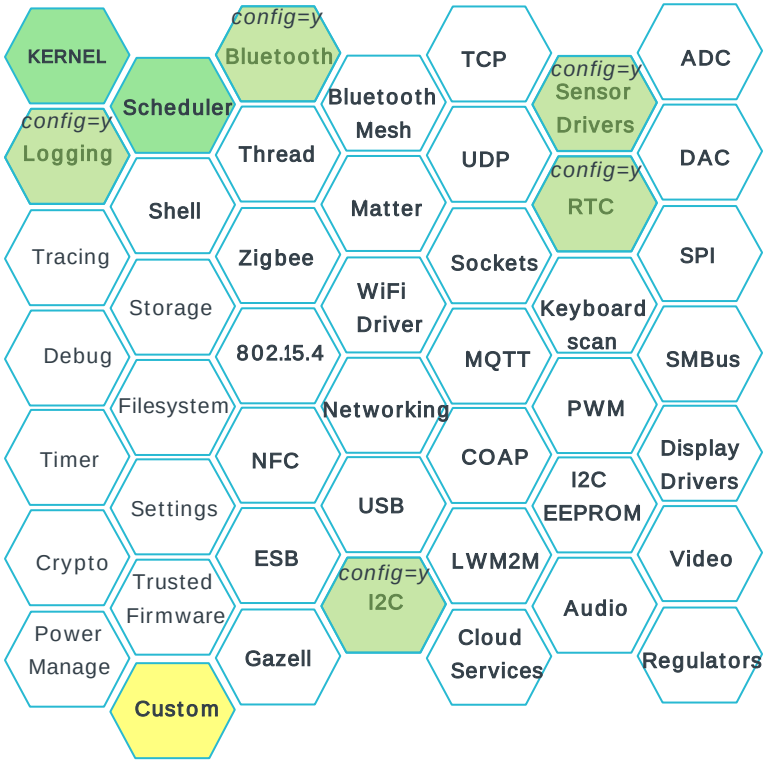
© Nordic Semiconductor

Configuration System (KCONFIG)

- Zephyr Kernel and Sub-System are configured at build time
  - Add software modules to your project
  - If a software module was added, further CONFIG symbols appear and allow you to configure the module
- Custom Kconfig is possible

- **Goal**: Configure software features without changing C source code

CONFIG_BT=y
CONFIG_BT_PERIPHERAL=y
CONFIG_BT_CENTRAL=n
...



© Nordic Semiconductor

Using Kconfig Symbols in Configuration Files

CONFIG_<symbol name>**=**<value>

There should be no spaces around the equal sign!

bool symbols can be enabled by setting them to **y** or **n**. For example:

CONFIG_BT=y

Other symbol types are assigned like this:

CONFIG_SOME_STRING="cool value"

CONFIG_SOME_INT=123

Comments use a "#".

For example:

 # This is a comment

Examples:

CONFIG_BT=y

CONFIG_BT_DEVICE_NAME=„MyBeacon"

CONFIG_I2C=y

CONFIG_I2C_SHELL=y

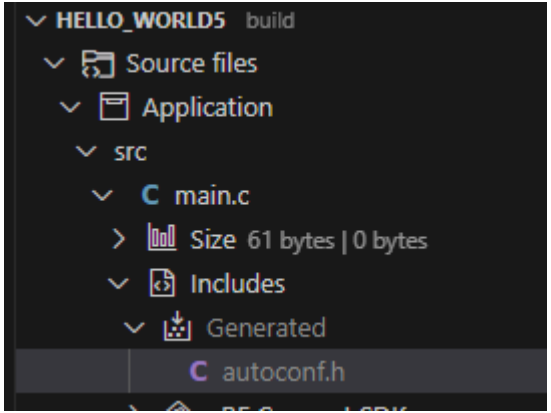
© Nordic Semiconductor

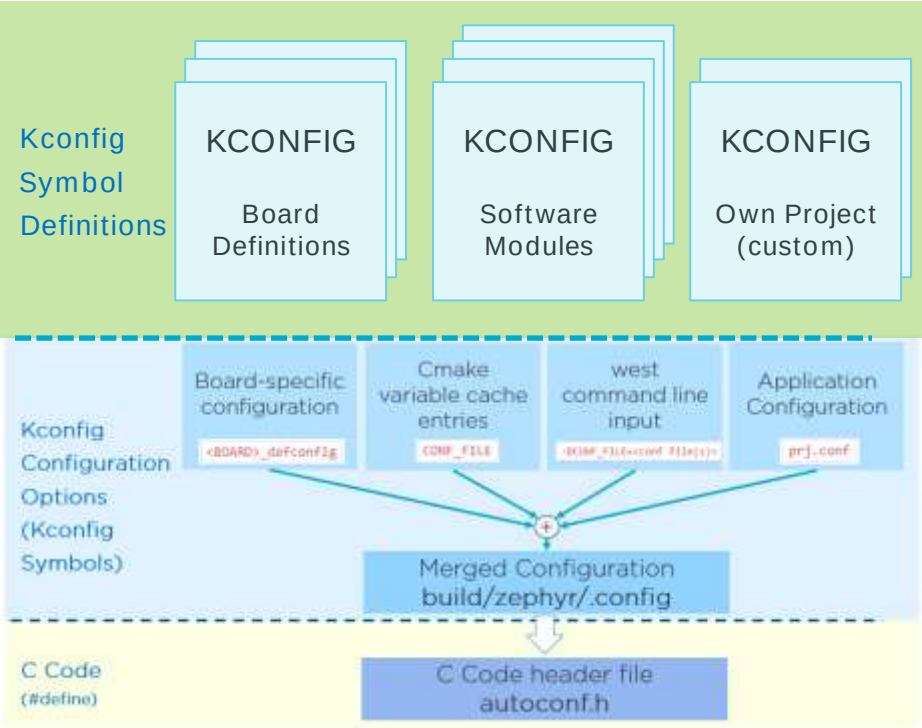
How does C Code understand KCONFIG?

- Build process creates a header file

./build/zephyr/include/generated/autoconf.h

- autoconf.h is automatically added to the project:



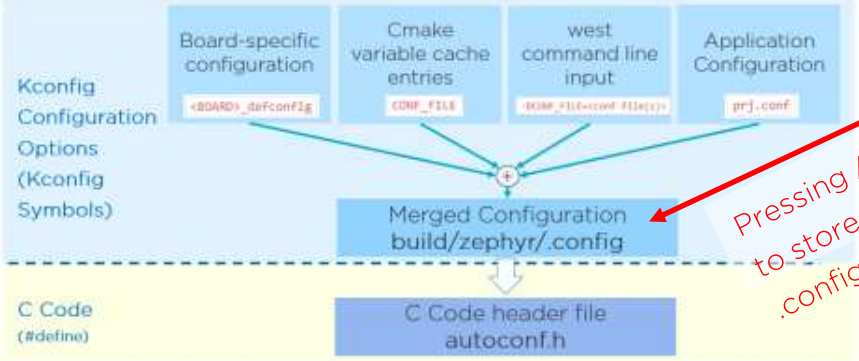


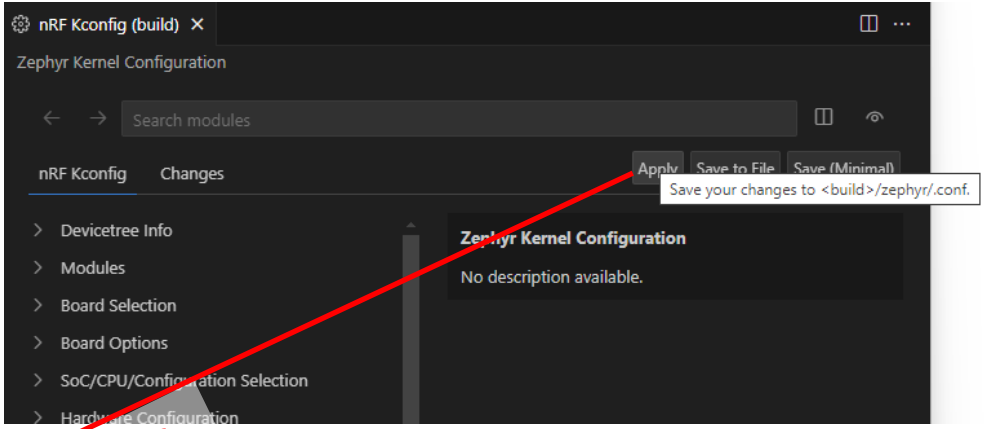
© Nordic Semiconductor

nRF Kconfig GUI

IMPORTANT when using KCONFIG Tools:
A pristine build will erase the .config settings!
=> add CONFIG settings in prj.conf

- Helpful tools:
  - nRF Kconfig GUI (default tool, included in VS Code environment)
  - guiconfig
  - menuconfig

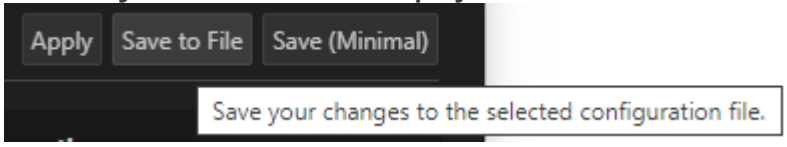




Pressing Apply causes to store changes in .config

NOTE:

Pressing **Save to File** stores changes into your selected prj.conf file



Copyright © 2025 Nordic Semiconductor. All rights reserved

© Nordic Semiconductor

DeviceTree – What is it?

- Defining Hardware Usage in separate files (DeviceTree file)
- These Symbols are then used in your Code
- Advantage:
  - Can easily be reused (Hardware can then adjusted by just changing the DeviceTree file)
  - Perfect for Device Driver
  - … but also usefull for your own Code
  - No Memory overhead due to macros

```
#include <devicetree.h>

#define LED0_NODE DT_ALIAS(led0)

#define LED0 DT_GPIO_LABEL(LED0_NODE, gpios)
#define PIN DT_GPIO_PIN(LED0_NODE, gpios)
#define FLAGS DT_GPIO_FLAGS(LED0_NODE, gpios)

void main(void){
   const struct device *dev;
   dev = device_get_binding(LED0);
   ret = gpio_pin_configure(dev, PIN,
                GPIO_OUTPUT_ACTIVE | FLAGS);
   ...
}
```

main.c

```
/ {
 leds {
  compatible = "gpio-leds";
  led0: led_0 {
  gpios = <&gpio0 13 GPIO_A
 };
```

DeviceTree Files

© Nordic Semiconductor

DeviceTree

- DeviceTree is a **hierarchical data structure** adopted from Linux Kernel **to describe hardware** (see „Devicetree Specification“, https://www.devicetree.org)
- Essential to the Zephyr device model: most devices are described in DeviceTree
- There are **two types of** DeviceTree **input files**:
  - the **sources** and the **bindings**, which describe the content
- DeviceTree in Zephyr is **parsed** at **compile time** and converted to **C header files**
- All DeviceTree information is accessible at **compile time** via the <zephyr/devicetree.h> API

© Nordic Semiconductor

DeviceTree Structure

Root node

Node (child of root node)

Sub-Node (child of *leds* node)

Node label (e.g. *led3*)

Porperty (e.g. *gpios*)

nrf54l15dk_common.dtsi

C > Nordic > NCS > v3.2.1 > zephyr > boards > nordic > nrf54l15dk > nrf54l15dk_common.dtsi > / > leds

```
/ {
    leds {
        compatible = "gpio-leds";

        led0: led_0 {
            gpios = <&gpio2 9 GPIO_ACTIVE_HIGH>;
            label = "Green LED 0";
        };

        led1: led_1 {
            gpios = <&gpio1 10 GPIO_ACTIVE_HIGH>;
            label = "Green LED 1";
        };

        led2: led_2 {
            gpios = <&gpio2 7 GPIO_ACTIVE_HIGH>;
            label = "Green LED 2";
        };

        led3: led_3 {
            gpios = <&gpio1 14 GPIO_ACTIVE_HIGH>;
            label = "Green LED 3";
        };
    };

    pwmleds {
```

© Nordic Semiconductor

DeviceTree Handling in blinky

❷ Get the Property „gpios“

❶ Point into DeviceTree Node

❸ Use DeviceTree settings to configure GPIO

DeviceTree File:

```
9   {
10      leds {
11          compatible = "gpio-leds";
12
13          led0: led_0 {
14              gpios = <&gpio2 9 GPIO_ACTIVE_HIGH>;
15              label = "Green LED 0";
16          };
```

```
76      aliases {
77          led0 = &led0;
```

C Code:

```
14  /* The devicetree node identifier for the "led0" alias. */
15  #define LED0_NODE DT_ALIAS(led0) ❶
16
17  /*
18   * A build error on this line means your board is unsupported.
19   * See the sample documentation for information on how to fix this.
20   */
21  static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpios); ❷
22
```

```
32      ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE); ❸
33      if (ret < 0) {
34          return 0;
```

© Nordic Semiconductor

How to obtain a Node Identifier

Examples on how to obtain the Node Identifier for `uart@abcd1234` node

```
/dts-v1/;
/ {
   chosen {
       zephyr,console = &uart0;
   };
   aliases {
      my-uart = &uart0;
   };
   soc {
      uart0: uart@abcd1234 {
          compatible = „vnd,soc-uart“;
          reg = <0xabcd1234 0x1000>;
          current-speed = <9600>;
      };
   };
};
```

```
/* by full path */
DT_PATH(soc, uart_abcd1234)

/* by node label */
DT_NODELABEL(uart0)

/* by using an alias */
DT_ALIAS(my_uart)

/* by using a choice */
DT_CHOSEN(zephyr_console)

/* by specifying an instance number of a
 * given compatible, e.g. 0 */
DT_INST(0, vnd_soc_uart)
```

© Nordic Semiconductor

Node Identifiers

Important

Non-alphanumeric characters like dash (-) and the at sign (@) in devicetree names are converted to underscores (_). The names in a DTS are also converted to lowercase.

https://docs.nordicsemi.com/bundle/ncs-latest/page/zephyr/build/dts/api-usage.html#node_identifiers

Example:

nrf52840dk_nrf52840.dts file

```
/* These aliases are provided for compatibility with samples */
aliases {
        led0 = &led0;
        led1 = &led1;
        led2 = &led2;
        led3 = &led3;
        pwm-led0 = &pwm_led0;
        sw0 = &button0;
        sw1 = &button1;
        sw2 = &button2;
        sw3 = &button3;
        bootloader-led0 = &led0;
        mcuboot-button0 = &button0;
        mcuboot-led0 = &led0;
        watchdog0 = &wdt0;
```

zephyr/samples/basic/blinky_pwm

```
#include <zephyr/drivers/pwm.h>

static const struct pwm_dt_spec pwm_led0 = PWM_DT_SPEC_GET(DT_ALIAS(pwm_led0));

#define MIN_PERIOD PWM_SEC(1U) / 128U
```

© Nordic Semiconductor

Accessing Properties from C Code

Examples on how to access DeviceTree Information

```
/dts-v1/;
/ {
   ...
     i2c0: i2c@abcd1234 {
       ...
       sensor0: sensor@ff {
          compatible = „vnd,mysensor“;
          reg = <0xff>;
          int-gpios = <&gpio0 1
                       GPIO_ACTIVE_LOW>;
          sample-freq = <1000>;
       };
     };
   };
};
```

```
/* use Node Label to obtain sensor node
 * identifier */
#define SENSOR_0 DT_NODELABEL(sensor0)
```

```
/* expands to 0xff */
DT_REG_ADDR(SENSOR_0)
```

```
/* expands to gpio() node identifier */
DT_PHANDLE(SENSOR_0, int_gpios)
/* equivalent (GPIO specific helpers) */
DT_GPIO_CTRL(SENSOR_0, int_gpios)
```

```
/* expands to 1 (pin cell) */
DT_PHA(SENSOR_0, int_gpios, pin)
/* equivalent (GPIO specific helpers) */
DT_GPIO_PIN(SENSOR_0, int_gpios)
```

```
/* expands to 1000 */
DT_PROP(SENSOR_0, sample_freq)
```

© Nordic Semiconductor

Most Important DeviceTree Properties

Property	
compatible	Name of the hardware device the node represents. Recommended format: ***vendor,device*** Build system uses compatible property to find the right bindings for the node.
reg	Information used to address the device. Its value is specific to a device. In general, it is a sequence of `(address, length)` pairs.
status	String describes whether the node is enabled. “okay” for enabled, and “disabled” for disabled.

© Nordic Semiconductor

DeviceTree Property Types

Property type	How to write	Example
string	Double quoted	`a-string = "hello, world!";`
int	between angle brackets (`<` and `>`)	`an-int = <1>;`
boolean	for `true`, with no value (for `false`, use `/delete-property/`)	`my-true-boolean;`
array	between angle brackets (`<` and `>`), separated by spaces	`foo = <0xdeadbeef 1234 0>;`
uint8-array	in hexadecimal *without* leading `0x`, between square brackets (`[` and `]`).	`a-byte-array = [00 01 ab];`
string-array	separated by commas	`a-string-array = "string one", "string two", "string three";`
phandle	between angle brackets (`<` and `>`)	`a-phandle = <&mynode>;`
phandles	between angle brackets (`<` and `>`), separated by spaces	`some-phandles = <&mynode0 &mynode1 &mynode2>;`
phandle-array	between angle brackets (`<` and `>`), separated by spaces	`a-phandle-array = <&mynode0 1 2>, <&mynode1 3 4>;`

© Nordic Semiconductor

DeviceTree Bindings

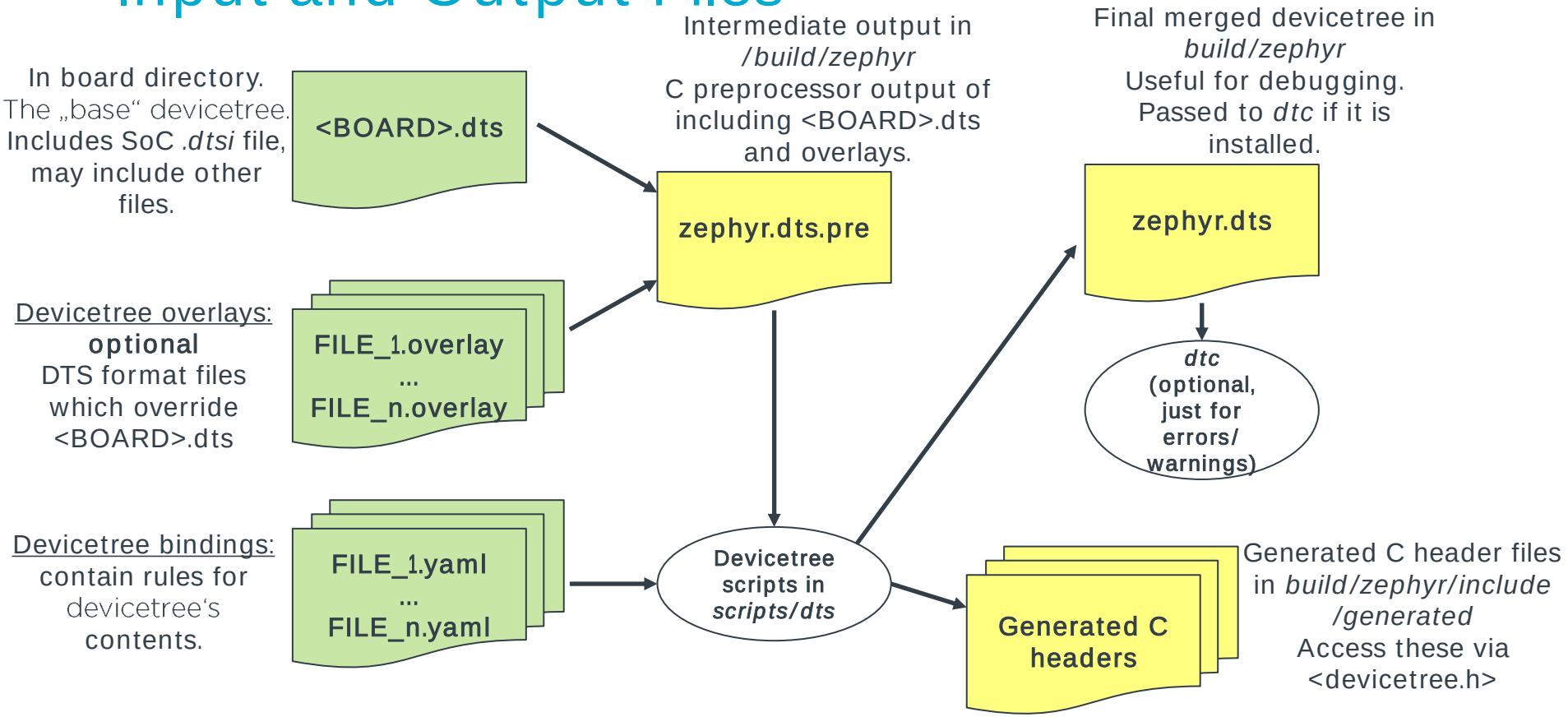
- **Content** of DeviceTree, including data types, is **described** in **binding** files
- Binding files are used by the DeviceTree parser to **validate content**
- Binding files are written in **YAML**, structure custom to Zephyr

Binding File:

```
descritpion: my dev driver
compatible: vnd,dev
include: base.yaml
properties:
   foo:
      type: int
      requried: true
```

DeviceTree Source File:

```
/ {
   dev0: dev@01234 {
      compatible = „vnd,dev“
      reg = <0x01234>;
      foo = <7>;
      };
   };
```

© Nordic Semiconductor

Input and Output Files